Uso de patrones de diseño de software: un caso práctico aplicado a Prosta-AI

Yimis Buenaventura, Juan David Riascos Erazo, Sergio Gabriel Meneses Silva,
Yeison Bolaños Ruiz y Fernando José Angulo Núñez

Facultad de Ingeniería, Programa de Ingeniería de Sistemas, Universidad Santiago de Cali

Gestión de Proyectos de TI

Ing. Jair Enrique Sanclemente Castro

22 de octubre de 2026

Resumen

Este informe presenta la aplicación de patrones de diseño de software al proyecto Prosta-AI, una aplicación web académica y experimental para cargar estudios de resonancia magnética de prueba, procesarlos con un modelo de segmentación 3D y visualizar la región de interés (ROI). Se explican el concepto y la descripción de un patrón y se seleccionan seis patrones del catálogo clásico: *Facade*, *Template Method*, *Strategy*, *Chain of Responsibility*, *State* y *Observer*. De cada uno se describen el propósito, la motivación, sus participantes y su aplicación en el proyecto, y se relaciona con los casos de uso, los requisitos de la matriz de trazabilidad y la arquitectura de software. La propuesta hace parte del entregable de diseño de la solución.

Palabras clave: patrones de diseño, arquitectura de software, segmentación de imágenes, trazabilidad de requisitos, Prosta-AI

Uso de patrones de diseño de software: un caso práctico aplicado a Prosta-AI

Introducción

Prosta-AI es una aplicación web académica y experimental que permite cargar estudios de resonancia magnética (MRI) de prueba, procesarlos con un modelo de segmentación 3D y visualizar la región de interés (ROI) sobre la imagen original. Este informe muestra cómo los patrones de diseño del catálogo clásico resuelven problemas concretos del proyecto, y los relaciona con los casos de uso (CU01 a CU06), con los requisitos de la matriz de trazabilidad (Equipo Prosta-AI, 2026b) y con la arquitectura de software del proyecto (Equipo Prosta-AI, 2026c).

El informe corresponde al paquete 1.3.2.2 de la estructura de desglose del trabajo (EDT), *Informe de patrones de sistemas aplicados*, y hace parte del entregable E3 (Diseño de la solución), previsto para el Ciclo 2 según el Acta de Constitución (Equipo Prosta-AI, 2026a). Para cada patrón se describen sus participantes y las clases que los cumplen en Prosta-AI; la implementación se hará en los ciclos de construcción.

El objetivo general es documentar la aplicación de patrones de diseño de software en la arquitectura de Prosta-AI. Los objetivos específicos son los siguientes:

- Explicar qué es un patrón de diseño y cómo se describe.
- Seleccionar los patrones que responden a problemas reales de Prosta-AI.
- Describir, para cada patrón, su propósito, su motivación, sus participantes y su aplicación en el proyecto.
- Relacionar cada patrón con los requisitos, los entregables y las pruebas de la matriz de trazabilidad.

Contexto del caso

Arquitectura

Prosta-AI usa una arquitectura cliente-servidor por capas, con un backend modular que corre en un solo proceso como mínimo viable (Equipo Prosta-AI, 2026c). Cuando el usuario pulsa «Procesar», la API no ejecuta la segmentación: registra un trabajo pendiente en

PostgreSQL, y una tarea local del mismo backend lo toma y ejecuta el procesamiento. La Tabla 1 resume las capas.

Tabla 1
Capas de la arquitectura de Prosta-AI

Capa	Tecnología	Responsabilidad
Frontend	React	Pantallas de CU01 a CU06, visor de la ROI y consulta del avance
API	FastAPI	Acceso, carga, inicio del procesamiento y consulta de resultados; registra los trabajos
Tarea de procesamiento	Python (mismo backend)	Toma los trabajos pendientes y ejecuta validación, preparación, segmentación y evaluación
Módulo de IA	PyTorch / MONAI	Preparación de la imagen e inferencia de segmentación 3D, desacoplado de la API
Datos	PostgreSQL y almacenamiento de archivos	Usuarios (solo alias y contraseña cifrada), estudios, trabajos, resultados y métricas, sin datos personales

Nota. Elaboración propia a partir de la arquitectura de software (Equipo Prosta-AI, 2026c).

Casos de uso

El único actor es el usuario académico. Los casos de uso son CU01 Acceder a la aplicación, CU02 Cargar estudios de MRI de prueba, CU03 Ejecutar el procesamiento automático, CU04 Visualizar y comparar la ROI, CU05 Consultar estudios y resultados registrados y CU06 Consultar orientación mediante agentes de IA (opcional).

Requisitos que condicionan el diseño

Los siguientes requisitos de la matriz de trazabilidad (Equipo Prosta-AI, 2026b) influyen directamente en las decisiones de diseño:

- Req 2.2: ningún dato que identifique a una persona queda almacenado; los metadatos identificables se eliminan o el archivo se rechaza.
- Req 2.3: toda salida se presenta como experimental y no diagnóstica.
- Req 3.2: el control de calidad y la preparación aplican las mismas transformaciones en entrenamiento e inferencia.
- Req 3.4: el módulo de IA está desacoplado del backend y se prefieren modelos

preentrenados con ajuste ligero.

- Req 3.5: el desempeño del modelo se mide con Dice e IoU en el conjunto de prueba.
- Req 4.1: carga de uno o varios estudios en formato DICOM o NIfTI, con validación de formato.
- Req 4.2: una vez el usuario pulsa «Procesar», el flujo completo se ejecuta sin otra intervención y la aplicación informa el estado de cada etapa.
- Req 4.5: los estudios procesados y sus resultados quedan registrados.
- Req 5.1: el procesamiento de un estudio toma dos minutos o menos.
- Req 5.4: el modelo y la aplicación son reproducibles, con configuración versionada y separada del código.

Selección de patrones

Se eligieron los patrones con cuatro criterios: que resuelvan un problema real del proyecto y se puedan trazar a un requisito; que sean simples de aplicar para un equipo de cinco estudiantes con una curva de aprendizaje exigente (riesgo 2 del Acta); que conserven el módulo de IA desacoplado; y que se puedan verificar con pruebas. La Tabla 2 resume la selección.

Tabla 2
Patrones seleccionados para Prosta-AI

Patrón	Familia	Problema que resuelve	CU	Req
Facade	Estructural	Ocultar la complejidad del procesamiento	CU03	3.4, 4.2
Template Method	Comportamiento	Garantizar el orden fijo de las etapas	CU03	3.2, 4.2
Strategy	Comportamiento	Intercambiar el modelo de segmentación	CU03	3.4, 5.4
Chain of Responsibility	Comportamiento	Encadenar las validaciones del archivo	CU02, CU03	2.2, 4.1
State	Comportamiento	Controlar el ciclo de vida de un trabajo	CU03, CU05	4.2, 4.5
Observer	Comportamiento	Registrar el avance de cada etapa	CU03	4.2

Nota. Elaboración propia. CU = caso de uso; Req = requisito de la matriz de trazabilidad.

Patrones aplicados

Facade (Fachada)

Propósito

Ofrecer una interfaz simple a un subsistema complejo.

Motivación

Procesar un estudio implica control de calidad, preparación con MONAI, inferencia del modelo, generación de la ROI y registro en la base de datos. Si la tarea de procesamiento llamara directamente a todas esas piezas, quedaría acoplada a los detalles del módulo de IA y cualquier cambio en el modelo obligaría a modificarla.

Participantes

La Tabla 3 muestra la fachada y los subsistemas que coordina.

Tabla 3
Participantes de la fachada de procesamiento

Rol en el patrón	Clase en Prosta-AI	Responsabilidad
Fachada	ProcessingService	Recibe el identificador del trabajo y coordina el procesamiento completo con una sola llamada
Subsistema de IA	Segmentation3DPipeline	Ejecuta las etapas de preparación, segmentación y generación de la ROI
Subsistema de datos	JobRepository	Obtiene el estudio y guarda el resultado y el estado en PostgreSQL
Cliente	ProcessingWorker	Toma los trabajos pendientes y llama a la fachada

Nota. Elaboración propia. Los nombres de las clases se escriben en inglés, como se hará en el código, y son propuestas de diseño.

Aplicación en Prosta-AI

El *endpoint* de CU03 solo registra el trabajo cuando el usuario pulsa «Procesar». La tarea local de procesamiento es quien usa `ProcessingService`, sin saber qué modelo se usa ni cuántas etapas tiene el flujo. Así se mantiene el módulo de IA desacoplado (Req 3.4) y el flujo se ejecuta de extremo a extremo con una sola llamada (Req 4.2).

Beneficios y cuidados

Reduce el acoplamiento y simplifica las pruebas de integración. Hay que evitar que la fachada acumule lógica de negocio hasta volverse una clase gigante.

Template Method (Método plantilla)

Propósito

Definir el esqueleto de un algoritmo en una clase base y dejar que las subclases redefinan ciertos pasos sin cambiar el orden general.

Motivación

El flujo siempre debe seguir el mismo orden: control de calidad, preparación, segmentación, generación de la ROI y, si existe una máscara de referencia, evaluación. Si cada desarrollador lo implementa a su manera, se pueden omitir pasos o cambiar su orden, y el Req 3.2 exige que las transformaciones sean las mismas en entrenamiento y en inferencia.

Participantes

La Tabla 4 presenta la clase base con el método plantilla y la subclase que completa el paso variable.

Tabla 4
Participantes del pipeline con método plantilla

Rol en el patrón	Clase en Prosta-AI	Responsabilidad
Clase abstracta	BasePipeline	Define el método `run`, que fija el orden de las etapas, e implementa los pasos comunes: control de calidad, preparación, generación de la ROI y evaluación cuando existe una máscara de referencia
Clase concreta	Segmentation3DPipeline	Completa el paso variable, la segmentación, delegándolo en la estrategia elegida

Nota. Elaboración propia. Los nombres de las clases se escriben en inglés, como se hará en el código, y son propuestas de diseño.

Aplicación en Prosta-AI

`BasePipeline` fija el orden de las etapas de CU03 y comparte el control de calidad y la preparación, de modo que entrenamiento e inferencia usan las mismas transformaciones. El paso de segmentación delega en la estrategia descrita en la siguiente sección.

Beneficios y cuidados

Evita duplicar código y asegura el mismo orden en todo el sistema. Si el esqueleto crece demasiado, se vuelve rígido; conviene mantener pocos pasos.

Strategy (Estrategia)

Propósito

Definir una familia de algoritmos, ponerlos en clases separadas y hacer que sus objetos sean intercambiables.

Motivación

El proyecto prevé modelos tipo U-Net o nnU-Net y prefiere modelos preentrenados con ajuste ligero por las limitaciones de cómputo (riesgo 1 del Acta). Es probable que el equipo pruebe más de un modelo, y cambiarlo no debería obligar a reescribir el flujo.

Participantes

La Tabla 5 muestra la interfaz de la estrategia, sus implementaciones y la clase que la utiliza.

Tabla 5
Participantes de la estrategia de segmentación

Rol en el patrón	Clase en Prosta-AI	Responsabilidad
Estrategia	SegmentationStrategy	Define cómo se segmenta un volumen: recibe el volumen preparado y devuelve la máscara 3D
Estrategias concretas	PretrainedUNetStrategy, NnUNetStrategy	Ejecutan la inferencia con cada modelo, respetando el mismo contrato de entrada y salida
Contexto	Segmentation3DPipeline	Usa la estrategia indicada en el archivo de configuración

Nota. Elaboración propia. Los nombres de las clases se escriben en inglés, como se hará en el código, y son propuestas de diseño.

Aplicación en Prosta-AI

La estrategia activa se elige desde un archivo de configuración versionado y separado del código, como pide el Req 5.4. Así se pueden comparar modelos con las métricas Dice e IoU (Req

3.5) sin tocar la API ni la interfaz.

Beneficios y cuidados

Facilita experimentar y reproducir resultados. Cada estrategia debe respetar el mismo contrato de entrada y salida.

Chain of Responsibility (Cadena de responsabilidad)

Propósito

Pasar una solicitud por una cadena de manejadores, donde cada uno decide si la procesa o la deja seguir.

Motivación

Un archivo debe cumplir varias condiciones: que su formato sea DICOM o NIfTI, que su tamaño sea válido, que no esté corrupto y que no contenga metadatos que identifiquen a una persona. Meter todas esas reglas en una sola función la vuelve larga, frágil y difícil de probar.

Participantes

La Tabla 6 presenta el manejador base y los eslabones de la cadena de validación.

Tabla 6
Participantes de la cadena de validación de archivos

Rol en el patrón	Clase en Prosta-AI	Responsabilidad
Manejador	Validator	Define la revisión de cada eslabón y enlaza con el siguiente; si una revisión falla, detiene la cadena con un mensaje claro
Manejadores concretos (admisión)	FormatValidator, SizeValidator	Comprueban que el archivo sea DICOM o NIfTI y que cumpla los límites de tamaño
Manejadores concretos (validación detallada)	IntegrityValidator, MetadataValidator	Comprueban que el archivo no esté corrupto y eliminan los metadatos identificables, sobre todo de las cabeceras DICOM, o rechazan el archivo
Cliente	UploadEndpoint (API) y ProcessingWorker	Arman la cadena correspondiente y la ejecutan

Nota. Elaboración propia. Los nombres de las clases se escriben en inglés, como se hará en el código, y son propuestas de diseño.

Aplicación en Prosta-AI

Siguiendo la arquitectura, la validación ocurre en dos momentos. En la admisión de CU02, la API ejecuta los eslabones de formato y tamaño, y cada uno devuelve un mensaje de error claro (Req 4.1). En la validación detallada de CU03, la tarea de procesamiento ejecuta los eslabones de integridad y metadatos; `MetadataValidator` elimina los datos identificables o rechaza el archivo (Req 2.2). Agregar una nueva regla equivale a agregar un eslabón.

Beneficios y cuidados

Cada regla se prueba por separado, con los archivos válidos, corruptos, de formato no soportado y con metadatos ficticios que pide la estrategia de pruebas de la matriz. El orden de la cadena debe definirse con cuidado.

State (Estado)

Propósito

Permitir que un objeto cambie su comportamiento cuando cambia su estado interno.

Motivación

Cada solicitud de procesamiento genera un trabajo que pasa por varias situaciones, y no todas las acciones son válidas en todas ellas. Por ejemplo, no se puede ver la ROI de un trabajo que aún no termina, ni iniciar de nuevo uno que ya está en proceso. Resolverlo con muchos condicionales dispersos genera errores.

Participantes

La Tabla 7 resume las transiciones permitidas, que coinciden con los estados de la arquitectura, y la Tabla 8 presenta los participantes.

Tabla 7
Transiciones del ciclo de vida de un trabajo

Estado actual	Acción	Estado siguiente
—	El usuario pulsa «Procesar»	Pendiente
Pendiente	La tarea local toma el trabajo	En proceso
En proceso	Todas las etapas terminan	Completado
En proceso	Una etapa falla (se registra cuál)	Fallido
Fallido	El usuario reintenta	Pendiente

Nota. Elaboración propia a partir de los estados definidos en la arquitectura de software (Equipo Prosta-AI, 2026c).

Tabla 8
Participantes de los estados de un trabajo

Rol en el patrón	Clase en Prosta-AI	Responsabilidad
Contexto	Job	Guarda su estado actual y delega en él las acciones
Estado	JobState	Define las acciones posibles y rechaza las transiciones inválidas
Estados concretos	PendingState, InProgressState, CompletedState, FailedState	Permiten solo las transiciones de la Tabla 7

Nota. Elaboración propia. Los nombres de las clases se escriben en inglés, como se hará en el código, y son propuestas de diseño.

Aplicación en Prosta-AI

El estado se guarda en PostgreSQL junto con cada trabajo (Req 4.5), se muestra en CU03 con el progreso y los errores por etapa, y aparece en la tabla de CU05.

Beneficios y cuidados

Las transiciones quedan explícitas y se pueden probar, incluidas las inválidas. Con pocos estados el patrón es sencillo; si no crecen, una enumeración simple también sería aceptable.

Observer (Observador)

Propósito

Definir un mecanismo de suscripción para notificar a varios objetos cuando ocurre algo en el objeto que observan.

Motivación

La aplicación debe informar el estado de cada etapa del procesamiento (Req 4.2). El pipeline no debería saber quién necesita esa información: la base de datos, un registro de tiempos o un registro de pruebas.

Participantes

La Tabla 9 presenta el sujeto y sus observadores.

Tabla 9
Participantes del sujeto y los observadores del progreso

Rol en el patrón	Clase en Prosta-AI	Responsabilidad
Sujeto	PipelineProgress	Mantiene la lista de observadores y les avisa del inicio, el fin o el error de cada etapa
Observador concreto	DatabaseRecorder	Guarda el estado de cada etapa en PostgreSQL, de donde lo lee la API
Observador concreto	TimingRecorder	Mide la duración de cada etapa para compararla con la meta de dos minutos

Nota. Elaboración propia. Los nombres de las clases se escriben en inglés, como se hará en el código, y son propuestas de diseño.

Aplicación en Prosta-AI

Cada etapa de `BasePipeline` notifica su inicio, su fin o su error. `DatabaseRecorder` conserva el avance en la base de datos y el frontend lo consulta mediante la API para mostrar la lista de etapas y el contador de tiempo de CU03. `TimingRecorder` aporta la evidencia de la meta de dos minutos del Req 5.1. Agregar un nuevo observador no exige modificar el pipeline.

Beneficios y cuidados

Desacopla el pipeline de quienes usan el progreso. Hay que evitar observadores lentos que retrasen el flujo.

Patrones complementarios

Los siguientes patrones también encajan en el proyecto, pero se proponen como opcionales según el avance de cada ciclo:

- *Decorator*: envolver cualquier salida o exportación con el aviso experimental y no

diagnóstico, sin modificar la lógica que genera el resultado (Req 2.3).

- *Adapter*: adaptar la interfaz de PyTorch/MONAI, o de los lectores de DICOM y NIfTI, a la que espera el backend, para conservar el desacople del módulo de IA.
- *Factory Method*: crear la estrategia de segmentación a partir del archivo de configuración, de modo que el resto del sistema no dependa de las clases concretas.
- *Singleton*: cargar los pesos del modelo una sola vez en memoria. Se propone con cuidado, porque introduce estado global y dificulta las pruebas.
- *Composite*: en React, la interfaz ya se compone como un árbol de componentes, lo que coincide con este patrón sin trabajo adicional.

Patrones no aplicados

Abstract Factory, *Builder*, *Prototype*, *Bridge*, *Flyweight*, *Mediator*, *Memento*, *Iterator*, *Command* y *Visitor* no se justifican con el alcance actual. Usarlos sería sobreingeniería, y el Acta fija un alcance académico y un tiempo reducido (Equipo Prosta-AI, 2026a). Pueden reconsiderarse si el alcance cambia mediante el control de cambios.

Trazabilidad con la matriz de requisitos

La Tabla 10 relaciona cada patrón con los requisitos que atiende, el entregable de la EDT y la evidencia de prueba prevista.

Tabla 10
Trazabilidad entre patrones, requisitos, entregables y pruebas

Patrón	Requisitos	Entregable	Evidencia de prueba
Facade	3.4, 4.2	E3, E6	Prueba de integración de extremo a extremo con estudios de prueba
Template Method	3.2, 4.2	E2, E6	Pruebas unitarias de las transformaciones; verificación del orden de las etapas
Strategy	3.4, 5.4	E4	Inferencia sobre casos de prueba con cada estrategia; cambio de estrategia por configuración
Chain of Responsibility	2.2, 4.1	E6	Archivos válidos, corruptos, de formato no soportado y con metadatos ficticios
State	4.2, 4.5	E6	Pruebas de transiciones válidas e inválidas; revisión de las tablas de PostgreSQL
Observer	4.2, 5.1	E6, E7	Cada etapa registra su estado y su duración, y el avance se refleja en CU03

Nota. Elaboración propia a partir de la matriz de trazabilidad de requisitos (Equipo Prosta-AI, 2026b).

Siguiendo el Req 6.4, cada *pull request* que implemente un patrón debe citar el identificador del requisito que atiende, y la matriz se actualiza al cerrar cada ciclo.

Consideraciones y riesgos

Un patrón se aplica solo si resuelve un problema concreto; si un diseño más simple basta, se prefiere el simple, para evitar la sobreingeniería. Por la curva de aprendizaje del equipo (riesgo 2), los patrones se introducen por ciclo, empezando por *Facade* y *Chain of Responsibility*, y se revisan en parejas. Los patrones tampoco deben agregar una latencia relevante: el tiempo total debe mantenerse en dos minutos o menos (Req 5.1) y se mide en las pruebas de rendimiento. Por último, la elección de cada patrón se registra como decisión técnica en el repositorio.

Conclusiones

Los patrones de diseño permiten que Prosta-AI sea más claro, flexible y fácil de probar. *Facade* y *Template Method* organizan el procesamiento, *Strategy* permite intercambiar el modelo sin romper el sistema, *Chain of Responsibility* ordena las validaciones y protege la privacidad, y *State* y *Observer* hacen visible el estado de cada trabajo. Todos se relacionan con un caso de uso, un requisito de la matriz y un componente de la arquitectura.

La propuesta debe confirmarse durante el Ciclo 2 (diseño de la solución) y ajustarse según lo que el equipo pueda implementar con el tiempo y las herramientas disponibles.

Referencias

Equipo Prosta-AI. (2026a). *Acta de constitución del proyecto Prosta-AI* (Versión 1.1) [Documento no publicado]. Universidad Santiago de Cali.

Equipo Prosta-AI. (2026b). *Matriz de trazabilidad de requisitos Prosta-AI* (Versión 1.1) [Documento no publicado]. Universidad Santiago de Cali.

Equipo Prosta-AI. (2026c). *Arquitectura de software de Prosta-AI* (Versión 1.1) [Documento no publicado]. Universidad Santiago de Cali.

Gamma, E., Helm, R., Johnson, R. y Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley.

Refactoring.Guru. (s. f.). *Patrones de diseño*. Recuperado el 22 de octubre de 2026, de `https://refactoring.guru/es/design-patterns`